Can we learn more when we cannot collect more data?

Counterfactual Residual Data Augmentation for Regression

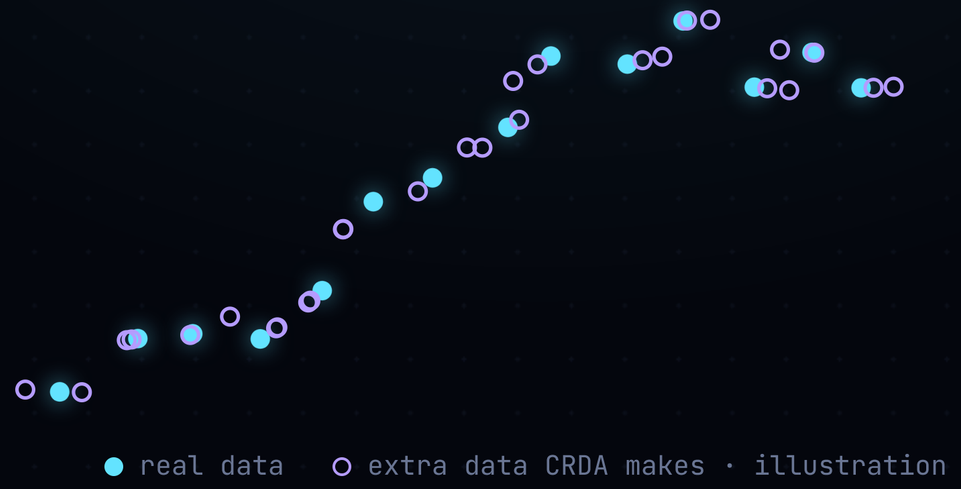
A way to make extra training data when you only have a little.

Hossein Mohebbi · Oliver Schulte · Ke Li · Pascal Poupart

University of Waterloo · Vector Institute · Simon Fraser University — **ICML 2026**, a leading machine-learning conference

We ran the authors' demo on 3 laptops · we have not yet repeated their main results

Anik · Alvi · Atul · Samim — Triple A — 23 October 2026



With only a little messy data, a model learns the wrong pattern

01 Only a few examples

02 Getting more is expensive

03 Measurements are noisy

REGRESSION

Predicting a *number*, such as how much heating a building needs.

NOISE

Random errors in the measurements.

RESIDUAL

The gap: how far off the model's guess was.



Illustration with made-up points

02 — WHY NOT JUST AUGMENT?

Copying a picture is easy. Copying a table row is not.

Pictures: a small rotation changes nothing



STILL A “7” ✓ so we get a free extra example

...but turn a 6 upside down and it becomes a 9

Tables: no safe trick like that

FLOOR AREA	BEDS	GARAGE	AREA	PRICE
92 m²	3	basic	zone B	£285,000
110 m²	3	basic	zone B	£285,000

SIMPLE COPY house 20% bigger, same price

✗ Wrong: we don’t know what a bigger house would cost

Illustration with made-up values

OTHER FIX 1

Blend existing rows together (SMOTE, C-Mixup: Chawla et al., 2002; Yao et al., 2022)

OTHER FIX 2

Make up rows with a learned model (CTGAN, TabDDPM: Xu et al., 2019; Kotelnikov et al., 2023)

WHAT THE AUTHORS SAY

these can help, but they lose each row’s *own* noise, and in the authors’ tests they sometimes made results worse

House row: made-up values · Other methods as described by Mohebbi et al. (2026), §2 and Table 2

Nudge one number. Keep the same mistake.

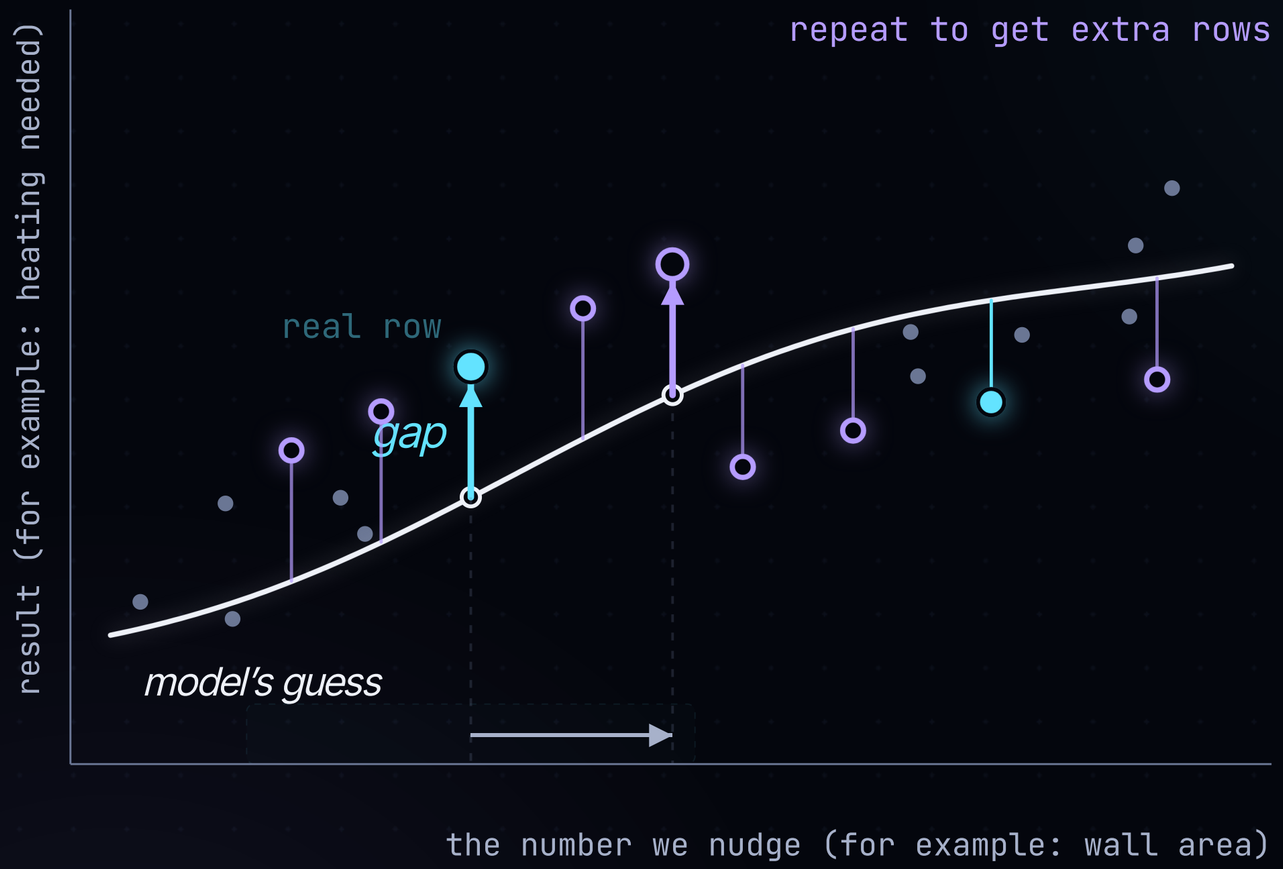


Illustration with made-up points, not data from the paper. The real method can nudge several inputs at once.

Think of a house: change the garage, keep the bidding war that pushed the price up.

1 — Train a normal model

$\hat{g}$ = the model's guess (a neural network or XGBoost)

2 — See how wrong it was on a real row

$$z = y - \hat{g}(x)$$

z = the gap (the residual)

3 — Nudge one number a little

$$x' = x \times (1 + \delta)$$

δ is random, at most ±10%. Shown: +9%. Everything else stays the same.

4 — Add the same gap back

$$y' = \hat{g}(x') + z$$

new label = new guess + the old gap

5 — Repeat to get extra rows

The new rows keep real-world noise.

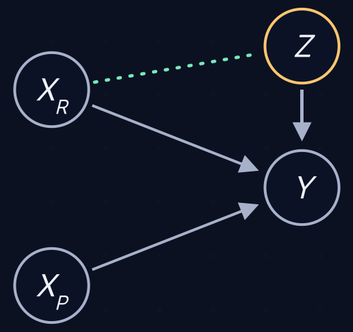
Method: Mohebbi et al. (2026), Algorithm 1 · ±10% is the authors' code default · Plot: illustration

It only works if the noise ignores the number we nudge

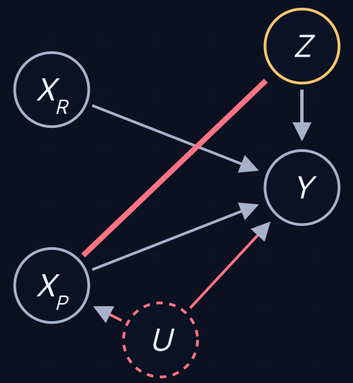
$$P(Z \mid X_P, X_R) = P(Z \mid X_R)$$

In plain words: once we know the other inputs, the noise tells us nothing about the number we nudge.

1 · An assumption — not a guarantee



✓ Fine: the number affects the result, not the noise.



✗ Broken: a hidden factor affects both, so the noise moves when we nudge.

→ affects ⋯ link that is allowed ━ link that breaks the rule

X_P number we nudge X_R other inputs Y result Z noise U hidden factor

after Fig. 1 of the paper

2 · TWO ROUGH CHECKS TO PICK SAFE NUMBERS

F1 ~~F2~~ F3 F4 ~~F5~~ F6 ~~F7~~ F8

illustration: example inputs

CHECK 1 (PC)
drop inputs that seem linked to the noise

CHECK 2
drop inputs that move together with the noise

what is left → safe to nudge

3 · A FINAL TEST

10 test rounds, with and without the extra rows

clearly better ($p < 0.05$) → use CRDA

otherwise → keep the plain model

$p < 0.05$ means under a 5% chance it's luck. Evidence, not a guarantee.

A weak starting model spoils the noise estimates, so the method can fail.

The two checks are rough guesses: they can miss hidden factors and curved links.

Our plan: check whether the authors' results hold up

- ✓ **Set up the authors' code**
  Python 3.11 and the exact library versions they list
- ✓ **Run their demo**
  on all three of our laptops
- 3 **Pick two small datasets**
  Energy Efficiency and Concrete Strength, with two models (a neural network and XGBoost)
- 4 **Compare plain vs CRDA**
  same data splits each time; 3 runs at first (the paper uses 15)
- 5 **Compare with the paper's table**
  do we see similar improvements?
- 6 **Write down every difference**
  problems, library versions, run times

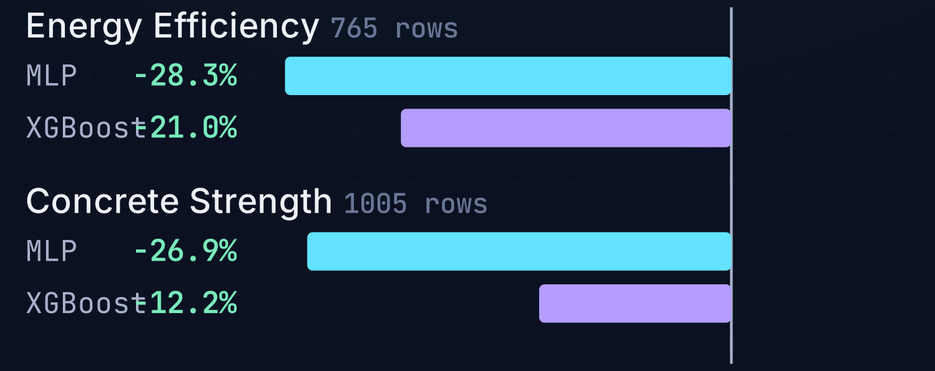
STEPS 1–2 DONE · STEPS 3–6 STILL TO DO

Results reported by the paper's authors — **not ours yet**

−22.9% neural network (MLP)

−6.4% XGBoost

average drop in error across 9 datasets



From the paper's main table · average of 15 runs · largest data size

TWO QUESTIONS FROM READING THEIR CODE (WE HAVE NOT CHECKED THEM YET)

? Their main table switches CRDA on even when the safety test says no
? Their final score averages 10 models, not one as the paper describes

Reported by Mohebbi et al. (2026): abstract and Table 1 · Code questions: authors' repo (experiments/*/config.json, src/experiment.py)

06 — OUR FIRST CHECK

The authors' demo runs on all three of our laptops

One run · one dataset · not the paper's main results

Each box: error before → after · −51.7% means the error roughly halves · p = chance the gain is just luck (under 0.05 passes)

LAPTOP	RUN TIME	NEURAL NETWORK (MLP)	XGBOOST
Authors' own saved result from their demo	—	0.00132 → 0.00064 **−51.7%** p = 0.00098	0.00130 → 0.00093 **−28.5%** p = 0.042
Anik M2 Apple M2 (8 cores) · 8 GB memory Graphics: Apple M2 (built in), not used macOS 14.6.1 · Python 3.11.5	41 s	0.00132 → 0.00064 **−51.7%** p = 0.00098 ✓ same as the authors'	0.00130 → 0.00093 **−28.5%** p = 0.042 ✓ same as the authors'
Alvi Ryzen 7 · RTX 3070 AMD Ryzen 7 5800H (8 cores) · 64 GB memory Graphics: NVIDIA RTX 3070 Laptop, not used Windows 11 (build 26340) · Python 3.11.15	3 min 04 s	0.00132 → 0.00064 **−51.7%** p = 0.00098 ✓ same as the authors'	0.00127 → 0.00095 **−24.8%** p = 0.032 different (plain model 2.7% lower)
Atul M5 Pro Apple M5 Pro · 24 GB memory Graphics: Apple M5 Pro (built in), not used macOS 27.0.1 · Python 3.11.17	34 s	0.00132 → 0.00064 **−51.7%** p = 0.00098 ✓ same as the authors'	0.00130 → 0.00093 **−28.5%** p = 0.042 ✓ same as the authors'

SAME LIBRARY VERSIONS ON ALL THREE

torch 2.7.0 · scikit-learn 1.7.0rc1 · xgboost 3.0.1 · numpy 2.2.6

PROBLEMS WE HIT

! A library version wouldn't run on new macOS; a newer one gave the same results

! Windows XGBoost differs, even for the plain model (0.00127 vs 0.00130); we don't know why yet

Each of us has one job, and someone double-checks it

A — PLANNED

Anik

Setting up the code

- Python 3.11 setup from the authors' exact versions
- Notes on what works on each kind of computer
- Records of settings and versions

ALSO CHECKS
Re-runs one of Alvi's experiments

PRESENTS
slides 1, 2, 3, 7 and this card

B — PLANNED

Alvi

Running the experiments

- Plain vs CRDA runs (neural network, XGBoost)
- Same data splits and settings every time
- Saves every settings and results file

ALSO CHECKS
Re-calculates Atul's comparison table from the raw results

PRESENTS
slides 4, 5, 7 and this card

C — PLANNED

Atul

Comparing and presenting

- Our results next to the paper's table
- List of problems and differences
- Slides, timing and rehearsal

ALSO CHECKS
Checks Samim's statistics

PRESENTS
slides 6, 7, 9, 10 and this card

D — PLANNED

Samim

Statistics and code checks

- Statistical tests on plain vs CRDA results
- Tests the two code questions we flagged
- Runs the second dataset (Concrete Strength)

ALSO CHECKS
Rebuilds Anik's setup on his own Windows laptop

PRESENTS
slides 9 and this card



Everyone: runs at least one experiment from start to finish · checks the final slides · practises to time

08 — TARGETS BEFORE WEEK 9

What we will show at the Week 9 update

Week 5 · today — Week 9 · progress update

1 — ✓ Done 22 Oct

Code set up and running

WE ARE DONE WHEN

The authors' demo runs on our laptops

PROOF

Exact versions and laptop details written down (3 laptops)

Anik

2 — by Week 7

Plain-model results saved

WE ARE DONE WHEN

Plain neural network and XGBoost results on dataset 1, with fixed settings

PROOF

Settings and results files for every run

Alvi

3 — by Week 8

First CRDA comparison

WE ARE DONE WHEN

Our improvement next to the paper's table

PROOF

Side-by-side table with p-values

Alvi Atul Samim

4 — Ongoing

Problem log

WE ARE DONE WHEN

Every difference or problem written down, with what we did

PROOF

Shared list, discussed at Week 9

Atul

OUR RESEARCH QUESTION

Do the authors' improvements hold up when we re-run their own code — and where do they not?

Weeks and owners agreed by the group · target 1 done (22 Oct); targets 2–4 planned, not started

REFERENCES

Where our information comes from

THE PAPER AND ITS CODE

1 Mohebbi, H., Schulte, O., Li, K. and Poupart, P. (2026) ‘Counterfactual residual data augmentation for regression’, *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, PMLR 306. Seoul, South Korea.

2 Mohebbi, H. *et al.* (2026) *CRDA: official code repository* [software]. GitHub. Available at: https://github.com/mhmohebbi/CRDA (Accessed: 19 October 2026).

3 Mohebbi, H. *et al.* (2026) *CRDA project page*. Available at: https://crda-project.github.io/ (Accessed: 19 October 2026).

OTHER METHODS WE MENTION

4 Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) ‘SMOTE: synthetic minority over-sampling technique’, *Journal of Artificial Intelligence Research*, 16, pp. 321–357.

5 Kotelnikov, A., Baranchuk, D., Rubachev, I. and Babenko, A. (2023) ‘TabDDPM: modelling tabular data with diffusion models’, *Proceedings of the 40th ICML*, PMLR 202, pp. 17564–17579.

6 Spirtes, P., Glymour, C. and Scheines, R. (2000) *Causation, Prediction, and Search*. 2nd edn. Cambridge, MA: MIT Press.

7 Wilcoxon, F. (1945) ‘Individual comparisons by ranking methods’, *Biometrics Bulletin*, 1(6), pp. 80–83.

8 Xu, L., Skoularidou, M., Cuesta-Infante, A. and Veeramachaneni, K. (2019) ‘Modeling tabular data using conditional GAN’, *Advances in Neural Information Processing Systems*, 32.

9 Yao, H., Wang, Y., Zhang, L., Zou, J.Y. and Finn, C. (2022) ‘C-Mixup: improving generalization in regression’, *Advances in Neural Information Processing Systems*, 35, pp. 3361–3376.

DATA WE USE

10 Tsanas, A. and Xifara, A. (2012) *Energy Efficiency* [dataset]. UCI Machine Learning Repository. Available at: https://archive.ics.uci.edu/dataset/242/energy+efficiency.

11 Yeh, I.-C. (1998) *Concrete Compressive Strength* [dataset]. UCI Machine Learning Repository. Available at: https://archive.ics.uci.edu/dataset/165/concrete+compressive+strength.

All URLs checked 19 October 2026